PTfinder.API - One-Page App Summary

Evidence source: code in this repository only (controllers, services, Program.cs, EF models).

What it is

PTfinder.API is an ASP.NET Core 8 backend for a PTfinder marketplace that manages coach profiles, availability, bookings, billing, gifts, and communications. It exposes REST endpoints plus a SignalR notifications hub, with SQL persistence through Entity Framework Core.

Who it is for

Primary persona: personal trainers/coaches using PTfinder to publish profiles, receive bookings, and manage subscriptions and payouts. Secondary users are clients booking sessions.

What it does

- Manages coach directory data and searchable discovery by category, speciality, location, gender, and sort options (`CoachesController`).
- Handles availability slots and booking lifecycle operations (create, fetch, status update, delete).
- Runs auth flows with email OTP verification and JWT-based proof tokens (`AuthVerificationController`, JWT config in `Program.cs`).
- Supports Stripe billing flows: Connect onboarding, subscription checkout/cancel, gift checkout, and webhook processing (`BillingController`).
- Sends transactional emails for OTP, booking events, gifts, and subscription events (`SmtpEmailSender`, `BookingEmailFlows`, billing email handlers).
- Stores profile/gallery media in Azure Blob Storage and returns time-limited read URLs (`BlobStorageService`, `GalleryMediaController`).
- Creates and pushes in-app notifications via DB + SignalR hub groups (`NotificationService`, `/hubs/notify`).

How it works

- Ingress layer: ASP.NET Core controllers under `Controllers/` expose API routes; SignalR hub at `/hubs/notify` handles realtime events.
- Business/services layer: billing, subscriptions, notifications, email, and blob storage services are registered in DI (`Program.cs`, `Services/`).
- Data layer: `AppDbContext` maps domain entities (Coach, Booking, Availability, Review, Notification, Partner, Gift, etc.) to SQL Server via EF Core.
- Infra integrations: Stripe (payments/connect/webhooks), Azure Blob Storage (media), SMTP (email), Hangfire + SQL storage (background jobs/dashboard).
- Flow: HTTP request -> controller -> service/EF Core -> SQL/third-party integrations -> response; selected events also emit SignalR notifications and email side effects.

How to run

- Prerequisites: .NET 8 SDK and SQL Server reachable by connection string `ConnectionStrings:mycon`.
- Set required configuration values (user-secrets, env vars, or appsettings): `ConnectionStrings:mycon`, `Stripe:SecretKey`, and `Jwt:Key` (startup fails fast if missing).
- Optional/feature-specific config: `Smtp:*`, `AzureStorage:*`, and Stripe plan/webhook settings for billing and media paths.
- From repo root: `dotnet restore` then `dotnet run --project PTfinder.API/PTfinder.API.csproj`.
- Open Swagger at `/swagger` on the launched URL (launch settings include `http://localhost:5197` and `https://localhost:7235`).

Not found in repo: explicit production deployment guide, frontend repository/link, and formal persona documentation.